

1. Poor Requirement Gathering

- **Stakeholder Identification**
 - ☐ Identify all relevant stakeholders (business owners, users, technical teams, etc.)
 - ☐ Ensure stakeholders are engaged early and throughout the project lifecycle.
 - **Clear Business Goals & Objectives**
 - ☐ Define and document the overall business goals the system is meant to achieve.
 - ☐ Ensure business requirements align with strategic goals.
 - **Detailed Functional & Non-Functional Requirements**
 - ☐ Collect both functional and non-functional requirements (e.g., performance, security, compliance).
 - ☐ Prioritize requirements based on business value.
 - **User Stories & Use Cases**
 - ☐ Develop detailed user stories or use cases for each requirement.
 - ☐ Validate user stories with actual end-users, if possible, to ensure they reflect real-world use.
 - **Prototype / Wireframes**
 - ☐ Create low-fidelity prototypes or wireframes to visualize requirements early.
 - ☐ Conduct review sessions with stakeholders to validate assumptions.
 - **Regular Communication & Feedback**
 - ☐ Hold frequent check-ins or workshops with stakeholders to clarify ambiguities.
 - ☐ Track changes to requirements and communicate them clearly.
-

2. Overlooking User Experience (UX)

- **User-Centered Design**
 - ☐ Involve real users early in the design process to ensure the product meets their needs.
 - ☐ Use personas to guide decisions about design and functionality.
- **Usability Testing**
 - ☐ Conduct usability testing throughout the design process, from wireframes to prototypes to final product.
 - ☐ Collect both qualitative and quantitative feedback from users.
- **Clear and Intuitive UI/UX Design**
 - ☐ Ensure the user interface is intuitive, consistent, and easy to navigate.
 - ☐ Follow established UI patterns and guidelines (e.g., Apple's Human Interface Guidelines, Material Design).
- **Mobile and Cross-Platform Compatibility**
 - ☐ Design with mobile-first principles or ensure a responsive design for multiple screen sizes and devices.
 - ☐ Conduct cross-browser testing for compatibility.

- **Accessibility Considerations**

- ☐ Follow WCAG (Web Content Accessibility Guidelines) to ensure the product is usable by people with disabilities.
 - ☐ Implement keyboard navigation, screen reader support, and color contrast adjustments.
-

3. Lack of Scalability in Architecture

- **Scalable Design Principles**

- ☐ Use a modular architecture to ensure future expansion and flexibility.
- ☐ Design for horizontal scalability (e.g., microservices, containerization) to handle increasing loads.

- **Cloud-Native Infrastructure**

- ☐ Consider using cloud services (e.g., AWS, Azure, GCP) for elasticity and resource management.
- ☐ Choose technologies that support auto-scaling, load balancing, and fault tolerance.

- **Database Scalability**

- ☐ Choose database architectures that can scale horizontally or vertically (e.g., sharded databases, NoSQL).
- ☐ Optimize for high availability and failover mechanisms.

- **Performance Optimization**

- ☐ Conduct performance testing early and identify bottlenecks in both infrastructure and application layers.
- ☐ Optimize both server-side and client-side code for speed and efficiency.

- **Capacity Planning**

- ☐ Conduct regular load testing to predict future traffic patterns.
 - ☐ Monitor system performance and plan for capacity increases.
-

4. Ignoring Cybersecurity Risks

- **Security by Design**

- ☐ Implement security best practices from the outset (e.g., secure coding, data encryption).
- ☐ Use frameworks that enforce security standards (e.g., OWASP Top Ten).

- **Risk Assessment & Threat Modeling**

- ☐ Perform a risk assessment and threat modeling to identify vulnerabilities early in the process.
- ☐ Prioritize threats based on likelihood and potential impact.

- **Data Protection & Privacy Compliance**

- ☐ Ensure adherence to relevant privacy regulations (e.g., GDPR, CCPA).
- ☐ Encrypt sensitive data both in transit and at rest.

- **Access Control & Authentication**
 - ☐ Implement least privilege access control to restrict data and system access.
 - ☐ Use multi-factor authentication (MFA) for user access.
 - **Regular Security Audits & Penetration Testing**
 - ☐ Perform regular security audits and penetration tests to identify weaknesses.
 - ☐ Remediate identified vulnerabilities in a timely manner.
 - **Secure Supply Chain Management**
 - ☐ Vet third-party libraries and software dependencies for security vulnerabilities.
 - ☐ Use software composition analysis tools to manage open-source risks.
-

5. Delayed Testing & QA

- **Shift Left Testing**
 - ☐ Integrate testing into every phase of the development lifecycle (e.g., unit testing, integration testing, acceptance testing).
 - ☐ Start testing early in the development process, not just at the end.
- **Automated Testing & Continuous Integration**
 - ☐ Set up a CI/CD pipeline that automates unit tests, integration tests, and deployments.
 - ☐ Implement automated UI tests to catch regressions in user interactions.
- **Test Coverage & Test Case Design**
 - ☐ Ensure comprehensive test coverage across all features and edge cases.
 - ☐ Design clear, repeatable test cases for both functional and non-functional testing.
- **Performance & Load Testing**
 - ☐ Conduct load, stress, and performance testing to identify scalability issues.
 - ☐ Simulate real-world user loads to evaluate system performance under stress.
- **User Acceptance Testing (UAT)**
 - ☐ Engage end-users in UAT to validate that the system meets their needs and expectations.
 - ☐ Resolve all UAT issues before moving to production.
- **Bug Tracking & Regression Testing**
 - ☐ Use a bug tracking system to log and prioritize defects.
 - ☐ Perform regression testing to ensure that fixes don't introduce new issues.